

Documentation For Train Tickets

Purchases

Summary:-

About the Data:

This project is built on a single UK train ticket transactions dataset, shared and analyzed collaboratively across five tools: Excel, Power BI, Tableau, Python, and Machine Learning. Each transaction record includes information about the payment (Payment Method, Price, Purchase Type, Time of Purchase), the ticket itself (Ticket Class, Ticket Type, Railcard, Refund Request), and the journey (Departure Station, Arrival Destination, Date of Journey, scheduled and actual arrival times, Journey Status, Duration, and Delay time).

Because the dataset covers so many angles — payments, pricing, stations, timing, and service reliability — the team split the analysis across tools and topics rather than duplicating the same work.

Together, the different parts of the project cover:

- Revenue and pricing: Total Revenue, Average Price, and how pricing relates to Purchase Type.
- Ticket behavior: Total Tickets sold, a breakdown by Railcard (Pie Chart), and a Tree Map crossing Ticket Type and Ticket Class to see which combinations are most common.
- Timing patterns: the average difference between scheduled and actual arrival time (i.e., average delay), the average time between ticket purchase and departure (how far in advance people book), and a line chart of trips by day to spot daily demand patterns.
- Station and route insights: Station Insights, a Grouped Column chart of trip counts by both Arrival Destination and Departure Destination, and a bar chart comparing Arrival Destination against Delayed/Cancelled journeys to flag problem routes.
- Payment and refund behavior: Payment Method breakdown, Refund Insights, and a Pie Chart of Refund Requests to understand how often and for what reasons refunds occur.
- Overall trends: a Line Chart of trip volume over time, and Ticket Class Insights to compare travel patterns across ticket categories.
- Machine Learning: predictive modeling built on top of these same fields (e.g. to estimate delays, cancellations, or pricing patterns) rather than purely descriptive charts.

In short, the dataset itself stayed constant, but each tool and teammate approached it from a different lens — some focused on financial metrics (revenue, pricing), others on operational reliability (delays, cancellations, refunds), and others on demand patterns (routes, stations, time trends) — giving a full 360° view of how passengers book, pay for, and experience UK train travel.

Excel (Meram):-

1- Data Cleaning:

The data was already cleaned just converted the time columns to their proper time type to ensure correct calculations and formatting across the report

2- Added New Columns:

Created new calculated column for the Journey duration using M language by subtracting the start (Departure time) from the end (Actual arrival time)

Created another column to calculate the delay duration by subtracting the scheduled arrival time (the time the train supposed to arrive at) from the Actual arrival time

After making sure everything is cleaned with no errors and ready for processing I did close and load

3-Pivot Tables:

- Created pivot tables in new worksheet to categorize and count total trips by status (Journey status analysis)

- Created pivot table and charts to map refund request percentages against journey statuses (showing refund rates for cancelled and delayed trips)

- Grouped data by months (Jan–Apr) to calculate total prices and revenues over time

- Summarized transaction counts per station to identify high-traffic hubs and travel volumes

- Looked at different ticket categories using simple charts and tables to see pricing and sales patterns

3- Dashboard design

- Color Palette: Used a cohesive and professional color palette hand-picked from the ColorHunt website

- KPI Cards: Displayed key metrics including Total Revenue (741,921), Total Tickets (31,653), Refund Rate (4%), Top Class (Standard), Top Destination (Manchester Piccadilly), and Ticket Preferences (Advance)

- Visual Charts: Included charts for Revenue by Ticket Type, Refund Requests donut chart, Journey Status distribution, Trips by Departure Station, Monthly Revenue Trend, Refund Rate by Journey Status, and Tickets Sold by Class

- Interactive Slicers: Added side filters for Departure Station, Ticket Type, Refund Request, Ticket Class, and Journey Status for dynamic data exploration

Power BI (Ahmed Adel):-

Cleaning and Formating:-

In Power Query, The data was pretty much clean, it just needed to change the type of the times columns to times and some values was negative which made the conversion has errors and that concluded to removing them boys shoes in remove errors from remove rows button.

New Columns, Measures and Dimensions:-

We needed a new column to calculate the duration of the journey by subtracting the end and the start and I did that with M language and also there was a time when a train actually arrives and when it should have arrived so we got the delay by subtracting the actual from the supposed one.

For the measures I wrote three dax formulas to calculate the average period of delay, the number of all trips and the average period from purchase day to Germany day to use them in the upcoming part. And the data was normalized so no need to do any normalization rather than the date table.

Choosing Colors and Picking Visuals :-

For the colours i used a color palette from [colorhunt](#) website and for the visuals and insights it was divided on us all so this is what I worked on :-

The number of all tickets, the average minutes of incoming trains delay, the most used departure station, pie chart of ticket types and a column chart between journey status and count of trips. And also I added a page useful time line charts like day timeline and months timeline in relation with number of tickets of course.

Tableau (Abrar):-

Data Cleaning (Power Query - Excel):-

First, I selected the whole table and did Remove Duplicates to make sure there were no repeated rows. Then I changed the data types of the columns. I also created two new columns and applied Trim on them, then changed their data type as well. After that, I checked if there were any errors or null values in the data. Once everything looked clean, I did Close and Load, then opened the file in Tableau and double checked that all the data types were correct before starting the visualization.

Tableau Dashboards:-

Dashboard 1: UK Train Usage Insights Purpose: shows general usage patterns - how people pay and travel.

- Payment Method (Pie Chart): I dragged Payment Method to Color and Label, and Count of Transaction ID to Angle, then changed the mark type to Pie. Insight: Credit Card is the most used (19,136), Debit Card the least (1,683).
- Station Insights (Text Table): I dragged Departure Station to Rows, then dragged Count of Transaction ID and other measures to Text using Measure Names/Measure Values so they show side by side. Insight: Manchester Piccadilly is the busiest departure station (5,650).
- Trips by Destination (Stacked Bar Chart): I dragged Arrival Destination to Columns, Count of Transaction ID to Rows, and Ticket Type to Color to split each bar by ticket type. Insight: Bristol Temple Meads is by far the busiest arrival destination (~8K), despite having almost no departures.
- Trips Over Time (Line Chart): I dragged Date of Journey to Columns and set it to Continuous by Week, then dragged Count of Transaction ID to Rows. The mark type changed to Line automatically, and I added a trend line. Insight: overall trend shows a slight decline despite weekly fluctuations.

Dashboard 2: Reliability & Routes Purpose: focuses on service quality and route-level demand.

- Journey Status (Pie Chart): I dragged Journey Status to Color and Label, and Count of Transaction ID to Angle, then set the mark type to Pie. Insight: the large majority of trips are On Time.
- Route Map (Heat Map): I dragged Departure Station to Rows and Arrival Destination to Columns, then dragged Count of Transaction ID to Color and Label, and set the mark type to Square. Insight: Manchester Piccadilly to Liverpool Lime Street is the single busiest route (4,628).
- Problem Stations (Bar Chart): I dragged Departure Station to Rows, added Journey Status to Filters and kept only Delayed and Cancelled, then dragged Count of Transaction ID to Columns as a bar. Insight: compared against the Route Map to check whether the busiest stations are also the most problematic ones.

Python (Mohamed Emad):-

1. Project Overview

In this project, I performed an end-to-end data analysis on the UK Train Rides dataset , containing 31,653 entries). The primary goal was to extract Key Performance Indicators (KPIs), analyze passenger behavior, evaluate train punctuality, and identify peak routes to build actionable insights for executive decision-making.

The analysis was executed in **Google Colab** using **Python** with **pandas** for data manipulation, alongside **matplotlib** and **seaborn** for visualization.

2. Data Cleaning & Feature Engineering

To prepare the dataset for analysis, I performed the following data preparation steps:

1. Handling Missing Values:

- Imputed missing entries in the **Railcard** column with 'None' to represent standard non-discounted tickets.

2. Data Type Conversions:

- Converted purchase and journey dates (**Date of Purchase**, **Date of Journey**) into standard **datetime** format.
- Converted departure, scheduled arrival, and actual arrival times (**Departure Time**, **Arrival Time**, **Actual Arrival Time**) into **timedelta** objects to enable mathematical operations.

3. Feature Creation:

- **Duration**: Calculated as scheduled arrival time minus departure time (**Arrival Time - Departure Time**).
- **Delay Time**: Calculated as actual arrival time minus scheduled arrival time (**Actual Arrival Time - Arrival Time**).
- **Booking Lead Time (Days)**: Derived as the difference in days between the journey date and purchase date (**Date of Journey - Date of Purchase**).

3. Key Performance Indicators (KPIs)

After data transformation, I computed the core metrics for the network:

- **Average Ticket Price: £23.44**
- **Total Revenue: £741,921.00**
- **Overall Average Delay: 3.25 minutes** (excluding cancelled flights/trips).
- **Average Delay (Delayed Trips Only): 42.21 minutes.**

4. Visualizations & Analytical Insights

1. Daily Trips Trend (Line Chart)

- **Visualization**: Time-series line chart displaying daily ride counts.
- **Insight**: Steady daily travel demand fluctuating mostly between **240 and 300 trips per day**. Significant sharp drops (outliers) occurred on specific dates (e.g., March 1st and April 1st), likely due to scheduled track maintenance or strike actions.

2. Railcard Usage Distribution (Pie Chart)

- **Visualization**: Proportional pie chart categorizing railcard usage.
- **Insight**:
 - **66.1%** of passengers purchase full-price tickets without discounts (None).
 - **15.3%** utilize Adult Railcards.
 - Disabled and Senior Railcards account for **9.8%** and **8.8%** respectively.

3. Booking Lead Time Distribution (Bar Chart)

- **Visualization**: Bar chart mapping the number of days between ticket booking and actual travel date.

- **Insight:** The vast majority of bookings occur on the same day (0 days) or within 1–2 days prior to travel. Advance bookings (7+ days prior) are minimal, highlighting an immediate-travel behavior pattern.

4. Top 5 Popular Routes (Horizontal Bar Chart)

- **Visualization:** Horizontal bar chart ranking combined origin and destination pairs.
- **Insight:**
 - The highest-volume route is **Manchester Piccadilly -> Liverpool Lime Street** with **4,628 trips**.
 - London hub stations (Euston, Kings Cross, Paddington, St Pancras) dominate the remaining top route spots.

5. Journey Status Distribution (Pie Chart)

- **Visualization:** Proportion of overall trip punctuality status.
- **Insight:**
 - **86.8%** of all scheduled trips arrived On Time.
 - **7.2%** experienced Delays.
 - **5.9%** were Cancelled, presenting an operational area for refund and reliability investigations.

Train Delay Prediction Using Machine Learning (Sara)

1. Project Overview

The goal of this project is to build a Machine Learning model that predicts the **train delay time in minutes** based on available journey and ticket information.

The target variable is:

Delay Time (minutes)

Since the target is a continuous numerical value, this is a **Regression problem**.

2. Data Preparation

The dataset contains information about train journeys, tickets, stations, journey dates, and arrival information.

The main features include:

- Price
- Date and time of journey
- Departure Station
- Arrival Destination
- Ticket Class
- Railcard
- Payment Method
- Purchase Type
- Ticket Type
- Journey Status
- Journey Duration

The target variable was created as:

Delay Time = Actual Arrival Time – Arrival Time

where the result represents the delay duration.

3. Exploratory Data Analysis

Before building the models, the dataset was explored to understand:

- Data types
- Missing values
- Duplicate values
- Numerical features
- Categorical features
- Target variable distribution
- Relationships between features and delay time

This step helped identify unsuitable columns and prepare the data for Machine Learning.

4. Feature Engineering

Several useful features were extracted from the journey date and time information.

The following features were created:

- Day of Journey
- Month of Journey
- Day of Journey
- Arrival Hour

- Departure Hour
- Actual Arrival Hour
- Journey Duration

The target **delay time** was originally stored as a **timedelta**, so it was converted into minutes:

```
y_train = y_train.dt.total_seconds() / 60  
y_test = y_test.dt.total_seconds() / 60
```

This made the target suitable for regression models.

5. Removing Data Leakage

Two features were identified as unsuitable for predicting delay before the journey outcome is known:

is_Delay

This feature was derived from the delay information itself, so using it would give the model direct information about the target.

Therefore, it was removed.

Journey Status

Journey Status contains values such as:

- Delayed
- On Time

This information can reveal the actual outcome of the journey. Therefore, using it to predict the delay can introduce **data leakage**.

For the realistic prediction model, **Journey Status** was removed.

6. Train-Test Split

The dataset was divided into training and testing sets using an 80/20 split.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,
```

```
y,  
test_size=0.2,  
random_state=42  
)
```

The training set was used to train the models, while the testing set was used to evaluate their performance on unseen data.

7. Handling Categorical Features

Several features were categorical, including:

- Arrival Destination
- Departure Station
- Ticket Class
- Railcard
- Payment Method
- Purchase Type
- Ticket Type

One-Hot Encoding was used to convert these categorical variables into numerical features.

```
pd.get_dummies(  
    X_train,  
    columns=categorical,  
    drop_first=True  
)
```

The training and testing feature sets were aligned to ensure that they contained the same columns.

8. Handling Missing Target Values

Some rows contained missing values in the target variable.

The corresponding rows were removed from both the training and testing sets.

After removing missing target values:

- Training samples: **23,798**
- Testing samples: **5,975**

9. Feature Scaling

StandardScaler was used to standardize the numerical feature values.

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train_clean)
```

```
X_test_scaled = scaler.transform(X_test_clean)
```

The scaler was fitted only on the training data and then applied to the test data.

10. Machine Learning Models

Several regression algorithms were tested:

1. Linear Regression
2. K-Nearest Neighbors Regressor
3. Support Vector Regression
4. Decision Tree Regressor
5. Random Forest Regressor
6. AdaBoost Regressor
7. Gradient Boosting Regressor

The models were evaluated using:

- MAE
 - MSE
 - RMSE
 - R² Score
-

11. Model Evaluation

the models produced the following results:

Model	MAE	MSE	RMSE	R ²
Random Forest	0.670	20.233	4.498	0.896

K-Nearest Neighbors	1.051	33.177	5.759	0.831
Gradient Boosting	1.314	33.825	5.815	0.827
Decision Tree	1.228	35.555	5.962	0.818
AdaBoost	1.502	47.519	6.893	0.757
Linear Regression	2.872	71.161	8.435	0.637
Support Vector Regression	1.486	72.720	8.527	0.629

The Random Forest model achieved the highest R^2 among the tested models on the leakage-free feature set.

Therefore, Random Forest was selected as the final model.

12. Final Random Forest Model

The final model was created using:

```
rf_model = RandomForestRegressor(  
    n_estimators=200,  
    random_state=42,  
    n_jobs=-1  
)  
  
rf_model.fit(X_train_scaled, y_train_clean)
```

Predictions were generated using:

```
y_pred = rf_model.predict(X_test_scaled)
```

The final performance was approximately:

- **MAE:** 0.67 minutes
- **RMSE:** 4.49 minutes
- **R^2 :** 0.896

Interpretation

An R^2 of approximately 0.896 means that the model explains about **89.6% of the variation in delay time** on the test data.

The MAE of approximately 0.67 minutes means that the average absolute prediction error was about 0.67 minutes.

13. Feature Importance

Random Forest provides feature importance values that indicate how much each feature contributed to the model's predictions.

The top features were:

Feature	Importance
Day of Journey	0.1936
Departure Station — Manchester Piccadilly	0.1117
Arrival Destination — London Euston	0.1095
Day of Journey	0.0941
Purchase Type — Station	0.0938
Month of Journey	0.0729
Price	0.0677
Arrival Destination — Leeds	0.0582
Payment Method — Credit Card	0.0461
Railcard — Senior	0.0371

Feature importance shows which features the Random Forest relied on more during its prediction process.

It does **not** mean that a feature directly causes train delays.

14. Actual vs Predicted Visualization

An Actual vs Predicted plot was used to visualize the model's predictions.

- X-axis: Actual Delay Time
- Y-axis: Predicted Delay Time

Points closer to the diagonal reference line represent predictions closer to the actual delay values.

15. Conclusion

A complete Machine Learning pipeline was developed to predict train delay time.

The main steps were:

Data Cleaning → EDA → Feature Engineering → Leakage Detection → Train/Test Split → Encoding → Scaling → Model Training → Model Evaluation → Model Selection → Feature Importance

Several regression models were compared, and Random Forest was selected as the final model based on its performance on the cleaned test data.

The final model achieved an R^2 of approximately **0.896** and an MAE of approximately **0.67 minutes**.